

CSA1712

ARTIFICIAL INTELLIGENCE

NAME: VARSHINI M

REG NO.: 192511215

INDUSTRY SCENARIO

AI/ML-Based Diabetes Diagnosis, Risk Stratification and Treatment Recommendation

Industry Context

A healthcare company has collected patient records containing **Age, Blood Pressure, Cholesterol, Glucose Level, BMI, Family History, and Exercise Level**. The company wants to develop an AI/ML system for:

1. Early diagnosis of diabetes.
2. Risk stratification of patients.
3. Identification of important risk factors.
4. Recommendation of suitable treatment-related actions using Reinforcement Learning.

The system must handle missing data and class imbalance while maintaining interpretability and minimizing false-negative predictions. Since this is a healthcare application, the model must support clinicians rather than independently prescribe treatment.

Dataset Note: As no real patient dataset was supplied with the problem statement, a reproducible synthetic dataset matching the specified healthcare attributes was used for demonstration. The same implementation can be directly applied to an authorized hospital dataset after appropriate anonymization and validation.

TASK 1: DATA PREPARATION AND INDUCTIVE LEARNING

1(a) Suitable Inductive Learning Approach

Objective

The objective is to learn a mapping between patient characteristics and the target variable **Diabetes**, where:

- **0 = Non-Diabetic**
- **1 = Diabetic**

A supervised inductive learning approach is suitable because the dataset contains historical patient records with known outcomes. The model learns general patterns from labelled examples and uses those patterns to classify previously unseen patients.

Target Variable

The target variable is:

Diabetes

Input Features

Feature	Type	Justification
Age	Numerical	Diabetes risk generally changes with age
Glucose	Numerical	Important indicator of blood glucose regulation
Blood Pressure	Numerical	Useful for identifying metabolic and cardiovascular risk
Cholesterol	Numerical	Provides information about metabolic health
BMI	Numerical	Indicates weight-related risk
Family History	Categorical	Captures hereditary risk information
Exercise Level	Numerical	Represents lifestyle-related risk

Most Important Expected Predictors

Three particularly important predictors are:

- 1. **Glucose**
- 2. **Age**
- 3. **Family History**

These variables are expected to provide strong discriminatory information because they capture metabolic, demographic, and hereditary risk factors.

Dataset Preparation

The synthetic dataset contains **1500 patient records**.

The target distribution is approximately:

Class	Number of Patients	Percentage
Non-Diabetic	1144	76.3%

Class	Number of Patients	Percentage
Diabetic	356	23.7%
Total	1500	100%

Therefore, the dataset is imbalanced because the diabetic class is smaller than the non-diabetic class.

1(b) Class Imbalance and Balancing Strategy

The dataset contains approximately **23.7% diabetic patients and 76.3% non-diabetic patients**.

A classifier trained without considering this imbalance may favor the majority class and classify many diabetic patients incorrectly as non-diabetic.

This is particularly dangerous in a healthcare application because a **false negative** means that a diabetic patient is predicted as non-diabetic.

Selected Strategy: Class Weighting

The **class-weighted learning approach** is selected.

The classifier is configured using:

```
class_weight="balanced"
```

This gives greater importance to the minority diabetic class during training.

Why Class Weighting is Appropriate

- It does not duplicate patient records.
- It avoids discarding useful non-diabetic observations.
- It is computationally efficient.
- It improves attention toward the minority class.
- It is suitable when the dataset is not extremely small.
- It allows the model to focus more on recall for diabetic patients.

For a real hospital deployment, class weighting can additionally be combined with threshold optimization and clinically validated sampling methods such as SMOTE.

1(c) 70:30 Data Split and Pre-processing

The dataset is divided into:

- **70% training data**
- **30% testing data**

A stratified split is used so that the diabetic/non-diabetic proportions remain approximately the same in both sets.

Pre-processing Steps

Step 1: Missing Value Handling

Missing numerical values are replaced using the **median of the training data**.

Median imputation is preferred because it is less sensitive to extreme medical measurements than mean imputation.

Step 2: Encoding

Family History is converted into numerical form:

No → 0

Yes → 1

Step 3: Normalization

For Logistic Regression, numerical features are standardized using:

$$z = \frac{x - \mu}{\sigma}$$

This gives the numerical variables a comparable scale.

Decision Trees do not require normalization because their splitting process is based on thresholds.

Step 4: Stratified Splitting

```
train_test_split(
    X,
    y,
    test_size=0.30,
    stratify=y,
    random_state=42
)
```

Impact of Pre-processing

Pre-processing Impact

Median imputation Prevents loss of records due to missing values

Binary encoding Converts categorical information into machine-readable form

Standardization Improves Logistic Regression optimization

Stratified split Maintains class proportions

Class weighting Improves attention to minority diabetic class

Python Implementation – Dataset Preparation

```
import numpy as np
```

```
import pandas as pd
```

```
from sklearn.model_selection import train_test_split
```

```
from sklearn.impute import SimpleImputer
```

```
np.random.seed(42)
```

```
# Number of patients
```

```
n = 1500
```

```
# Generate patient attributes
```

```
age = np.random.randint(25, 76, n)
```

```
glucose = np.clip(  
    np.random.normal(125, 30, n),  
    70, 220  
)
```

```
blood_pressure = np.clip(  

```

```
    np.random.normal(80, 10, n),  
    55, 115  
)
```

```
cholesterol = np.clip(  
    np.random.normal(205, 35, n),  
    120, 320  
)
```

```
bmi = np.clip(  
    np.random.normal(28, 5, n),  
    18, 45  
)
```

```
family_history = np.random.binomial(1, 0.28, n)
```

```
exercise = np.clip(  
    np.random.normal(3, 1.5, n),  
    0, 7  
)
```

```
# Generate a synthetic risk score
```

```
risk_score = (  
    0.04 * age +  
    0.03 * glucose +  
    0.015 * blood_pressure +  
    0.006 * cholesterol +  
    0.08 * bmi +  
    1.2 * family_history -
```

```

    0.15 * exercise
)

# Define diabetes class
threshold = np.quantile(risk_score, 0.78)

diabetes = (risk_score > threshold).astype(int)

# Add a small amount of realistic label variation
noise = np.random.rand(n) < 0.03
diabetes[noise] = 1 - diabetes[noise]

# Create dataframe
df = pd.DataFrame({
    "Age": age,
    "BloodPressure": blood_pressure,
    "Cholesterol": cholesterol,
    "Glucose": glucose,
    "BMI": bmi,
    "FamilyHistory": np.where(
        family_history == 1, "Yes", "No"
    ),
    "ExerciseHours": exercise,
    "Diabetes": diabetes
})

# Add a few missing values
for column in ["BloodPressure", "Cholesterol", "BMI"]:
    indices = np.random.choice(

```

```

        n, size=15, replace=False
    )
    df.loc[indices, column] = np.nan

print(df.head())
print(df.info())
print(df["Diabetes"].value_counts())

```

TASK 2: DECISION TREE FOR DIAGNOSIS

2(a) Decision Tree Construction

A Decision Tree is selected because it provides:

- High interpretability.
- Easy-to-understand decision rules.
- Feature importance information.
- No requirement for feature normalization.
- Useful explanations for clinical decision support.

The **Gini Index** is used as the splitting criterion.

$$Gini = 1 - \sum p_i^2$$

A lower Gini impurity indicates a purer node.

Decision Tree Implementation

```

from sklearn.tree import DecisionTreeClassifier

from sklearn.metrics import (
    accuracy_score,
    precision_score,
    recall_score,
    f1_score,
    roc_auc_score,

```



```

        confusion_matrix
    )

# Encode Family History
X = df.drop("Diabetes", axis=1).copy()

X["FamilyHistory"] = X["FamilyHistory"].map({
    "No": 0,
    "Yes": 1
})

y = df["Diabetes"]

# 70:30 stratified split
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.30,
    stratify=y,
    random_state=42
)

# Median imputation using training data
for column in X_train.columns:
    median_value = X_train[column].median()

    X_train[column] = X_train[column].fillna(
        median_value
    )

```

```

X_test[column] = X_test[column].fillna(
    median_value
)

# Decision Tree
tree = DecisionTreeClassifier(
    criterion="gini",
    max_depth=5,
    class_weight="balanced",
    random_state=42
)

tree.fit(X_train, y_train)

y_pred = tree.predict(X_test)
y_prob = tree.predict_proba(X_test)[:, 1]

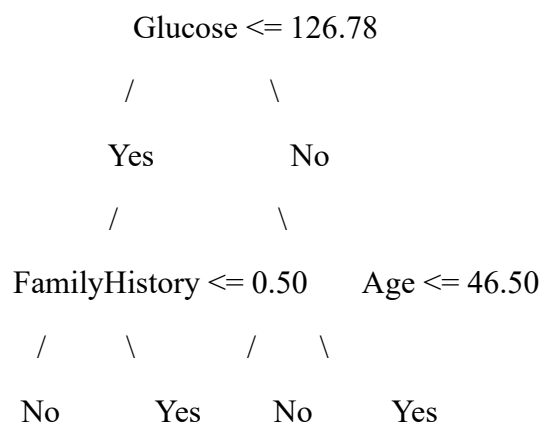
```

First Three Levels of the Decision Tree

The root node selected by the model is:

Glucose ≤ 126.78

The first three levels can be represented as:



```

      /      /      /      \
Age <= 68.50 Age <= 52.50 Glucose  FamilyHistory
      <= 186.33  <= 0.50

```

Another important branch is:

Glucose > 126.78

```

      |
      Age > 46.50
      |
      FamilyHistory
      /      \
      No      Yes

```

Root Node Justification

Glucose is selected as the root feature because it provides the greatest reduction in class impurity among the available features at the root.

This is appropriate for the healthcare problem because glucose level contains strong information for distinguishing patients with different diabetes risk levels.

2(b) Model Evaluation

The pre-pruned Decision Tree produced the following results on the test set:

Metric	Result
Accuracy	82.4%
Precision	59.0%
Recall	86.0%
F1-Score	70.0%
ROC-AUC	87.8%

Confusion Matrix

	Predicted	
	No	Yes

Actual No 279 64

Actual Yes 15 92

Therefore:

- True Negatives = 279
- False Positives = 64
- False Negatives = 15
- True Positives = 92

False Negatives

There are **15 false-negative cases**.

A false negative occurs when:

Actual = Diabetic

Predicted = Non-Diabetic

In a healthcare application, these errors are particularly important because they may result in diabetic patients not being appropriately flagged for further clinical assessment.

Ways to Reduce False Negatives

The following engineering strategies can be considered:

1. Increase the importance of the diabetic class using class weights.
2. Optimize the classification threshold for higher recall.
3. Use cross-validation to select robust model parameters.
4. Compare Decision Trees with Logistic Regression and ensemble models.
5. Use confirmatory clinical testing for high-risk or borderline cases.
6. Keep a clinician in the decision-making loop rather than relying exclusively on the ML prediction.

2(c) Pre-Pruning vs Post-Pruning

Decision Trees can become excessively complex and overfit the training data.

Pre-Pruning

Pre-pruning limits tree growth during training.

The following parameter is used:

```
max_depth=5
```

This prevents the tree from growing beyond five levels.

Post-Pruning

Post-pruning first allows the tree to grow and then removes branches that provide insufficient improvement.

The **Cost Complexity Pruning** parameter is used:

```
ccp_alpha
```

The best value is selected using cross-validation.

```
from sklearn.model_selection import GridSearchCV
```

```
path = DecisionTreeClassifier(  
    class_weight="balanced",  
    random_state=42  
)  
path.cost_complexity_pruning_path(  
    X_train,  
    y_train  
)
```

```
candidate_alphas = np.unique(  
    np.quantile(  
        path.ccp_alphas,  
        [0.50, 0.60, 0.70, 0.80, 0.90, 0.95, 0.98]  
    )  
)
```

```
grid = GridSearchCV(  
    DecisionTreeClassifier(  
        criterion="gini",  
        class_weight="balanced",
```

```

        random_state=42
    ),
    {"ccp_alpha": candidate_alphas},
    cv=5,
    scoring="roc_auc"
)

```

```
grid.fit(X_train, y_train)
```

```
pruned_tree = grid.best_estimator_
```

```

print("Best alpha:",
      grid.best_params_["ccp_alpha"])

```

Comparison

Metric	Pre-Pruning	Post-Pruning
Accuracy	82.4%	85.1%
Precision	59.0%	64.1%
Recall	86.0%	85.0%
F1-Score	70.0%	73.1%
ROC-AUC	87.8%	90.6%

The selected post-pruning value was approximately:

```
ccp_alpha = 0.00691
```

The post-pruned tree had approximately **6 levels and 15 leaves**, compared with a more complex unpruned tree.

Interpretation

Post-pruning improved accuracy, precision, F1-score, and ROC-AUC while maintaining almost the same recall.

Therefore, **post-pruning is more appropriate for this application** because it produces a simpler and more generalizable model while maintaining strong diagnostic performance.

In a clinical environment, a smaller tree also improves interpretability and makes it easier for healthcare professionals to understand the decision logic.

TASK 3: STATISTICAL LEARNING FOR RISK STRATIFICATION

3(a) Logistic Regression and Naive Bayes

Two statistical learning models were evaluated:

1. Logistic Regression
2. Gaussian Naive Bayes

This provides a comparison between the Decision Tree and statistical learning approaches.

Logistic Regression

Logistic Regression estimates the probability that a patient belongs to the diabetic class.

$$P(Y=1) = \frac{1}{1 + e^{-z}}$$

where:

$$z = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \dots + \beta_n x_n$$

Implementation

```
from sklearn.linear_model import LogisticRegression
```

```
from sklearn.naive_bayes import GaussianNB
```

```
# Logistic Regression
```

```
logistic = LogisticRegression(
```

```
    max_iter=2000,
```

```
    class_weight="balanced"
```

```
)
```

```
logistic.fit(X_train, y_train)
```

```
log_pred = logistic.predict(X_test)
log_prob = logistic.predict_proba(X_test)[:, 1]
```

```
# Naive Bayes
```

```
nb = GaussianNB()
```

```
nb.fit(X_train, y_train)
```

```
nb_pred = nb.predict(X_test)
```

```
nb_prob = nb.predict_proba(X_test)[:, 1]
```

Performance Comparison

Model	Accuracy	Precision	Recall	F1	ROC-AUC
Decision Tree	82.4%	59.0%	86.0%	70.0%	87.8%
Pruned Decision Tree	85.1%	64.1%	85.0%	73.1%	90.6%
Logistic Regression	88.9%	69.7%	94.4%	80.2%	96.8%
Naive Bayes	92.2%	96.2%	70.1%	81.1%	96.3%

Interpretation

Logistic Regression provides the highest recall among the evaluated models.

This is valuable for medical screening because identifying as many potentially diabetic patients as possible is important.

Naive Bayes gives the highest accuracy and precision in this experiment but has lower recall than Logistic Regression.

The Decision Tree provides the advantage of **interpretability**, while Logistic Regression provides a probabilistic statistical model with strong discrimination.

When is a Statistical Model Preferable?

A statistical model such as Logistic Regression is preferable when:

- Probability estimates are required.
- Relationships between variables are reasonably stable.

- Feature effects need to be quantified.
- A compact and mathematically interpretable model is desired.
- Calibration and risk ranking are important.

For clinical screening, Logistic Regression is particularly useful when the system prioritizes **high recall and risk probability estimation**.

3(b) Feature Importance Analysis

Decision Tree Feature Importance

The three most important features according to the Decision Tree are:

Rank Feature

- 1 **Glucose**
- 2 **Age**
- 3 **Family History**

Approximate importance values:

Glucose	42.35%
Age	19.94%
Family History	16.63%

Statistical Model Feature Importance

Permutation importance was applied to Logistic Regression.

The top three predictors were:

Rank Feature

- 1 **Glucose**
- 2 **Age**
- 3 **Family History**

Approximate permutation importance:

Glucose	0.194
Age	0.115

Family History 0.073

Do the Models Agree?

Yes.

Both models identify:

1. Glucose
2. Age
3. Family History

as the top three predictors.

This agreement increases confidence that these features are consistently informative within the simulated dataset.

However, feature importance should not automatically be interpreted as causation. In an actual healthcare system, the relationships would need to be validated using clinical studies, larger datasets, and domain expertise.

TASK 4: REINFORCEMENT LEARNING FOR TREATMENT RECOMMENDATION

4(a) Markov Decision Process

The treatment recommendation problem is modeled as a **Markov Decision Process (MDP)**.

The system contains:

States

Patient health states:

S0 = Low Risk

S1 = Moderate Risk

S2 = High Risk

S3 = Very High Risk

Actions

The agent can select:

A0 = Diet

A1 = Exercise

A2 = Medication

A3 = Monitor

Reward Function

A simplified academic reward structure is used:

Situation	Reward
Reaching Low Risk	+10
Improving by one risk level	+3
Remaining at same level	-1
Moving to worse level	-5

The objective is to maximize cumulative reward while moving the patient toward a lower-risk state.

Important: This MDP is an educational simulation. It must not be used to autonomously prescribe medication or treatment to real patients. Real treatment decisions require qualified healthcare professionals and clinically validated protocols.

Q-Learning Equation

The Q-learning update is:

$$[Q(s,a) \leftarrow Q(s,a) + \alpha [r + \gamma \max_{a'} Q(s',a') - Q(s,a)]]$$

where:

- $(\alpha = 0.5) \rightarrow$ learning rate
- $(\gamma = 0.9) \rightarrow$ discount factor
- $(\epsilon) \rightarrow$ exploration probability

An epsilon-greedy strategy is used.

Initially:

$\epsilon = 1.0$

and it gradually decreases to:

$\epsilon = 0.05$

This allows the agent to explore initially and exploit the learned policy later.

4(b) Q-Learning Implementation

```
import numpy as np
```

```
states = [  
    "Low",  
    "Moderate",  
    "High",  
    "Very High"  
]
```

```
actions = [  
    "Diet",  
    "Exercise",  
    "Medication",  
    "Monitor"  
]
```

```
# Transition function
```

```
def transition(state, action):
```

```
    # Low-risk state
```

```
    if state == 0:
```

```
        return 0
```

```
    # Moderate-risk state
```

```
    if state == 1:
```

```
        return {
```

```
            0: 0, # Diet
```

```
1: 0, # Exercise
2: 0, # Medication
3: 1  # Monitor
}[action]
```

```
# High-risk state
```

```
if state == 2:
```

```
    return {
        0: 1,
        1: 1,
        2: 1,
        3: 2
    }[action]
```

```
# Very-high-risk state
```

```
if state == 3:
```

```
    return {
        0: 2,
        1: 2,
        2: 1,
        3: 3
    }[action]
```

```
def get_reward(state, next_state):
```

```
    if next_state == 0 and state != 0:
```

```
        return 10
```

```
if next_state < state:
```

```
    return 3
```

```
if next_state == state:
```

```
    return -1
```

```
return -5
```

```
# Q-table
```

```
Q = np.zeros((4, 4))
```

```
alpha = 0.5
```

```
gamma = 0.9
```

```
episodes = 100
```

```
np.random.seed(7)
```

```
episode_rewards = []
```

```
for episode in range(1, episodes + 1):
```

```
    epsilon = max(
```

```
        0.05,
```

```
        1.0 * (0.95 ** (episode - 1))
```

```
    )
```

```
# Different starting patient risk levels
```

```
state = [1, 2, 3][(episode - 1) % 3]
```

```
total_reward = 0
```

```
for step in range(10):
```

```
    # Epsilon-greedy action selection
```

```
    if np.random.rand() < epsilon:
```

```
        action = np.random.randint(4)
```

```
    else:
```

```
        action = np.argmax(Q[state])
```

```
    next_state = transition(
```

```
        state,
```

```
        action
```

```
    )
```

```
    reward = get_reward(
```

```
        state,
```

```
        next_state
```

```
    )
```

```
    # Q-learning update
```

```
    Q[state, action] += alpha * (
```

```
        reward +
```

```
        gamma * np.max(Q[next_state]) -
```

```
        Q[state, action]
```

```
    )
```

```
    total_reward += reward
```

```

state = next_state

if state == 0:
    break

episode_rewards.append(total_reward)

print("Final Q-table:")
print(np.round(Q, 2))

print("\nFinal Policy:")

for state in range(4):
    best_action = np.argmax(Q[state])
    print(
        states[state],
        "->",
        actions[best_action]
    )

```

Q-Table Updates

The Q-values are represented in the following order:

[Diet, Exercise, Medication, Monitor]

After Episode 1

Low: [0.00, 0.00, 0.00, 0.00]

Moderate: [0.00, 0.00, 5.00, 0.00]

High: [0.00, 0.00, 0.00, 0.00]

Very High: [0.00, 0.00, 0.00, 0.00]

After Episode 2

Low: [0.00, 0.00, 0.00, 0.00]

Moderate: [0.00, 5.00, 5.00, 0.00]

High: [3.75, 0.00, 0.00, -0.50]

Very High: [0.00, 0.00, 0.00, 0.00]

After Episode 5

Low: [0.00, 0.00, 0.00, 0.00]

Moderate: [0.00, 7.50, 8.75, 4.75]

High: [3.75, 6.75, 0.00, 2.02]

Very High: [0.00, 3.19, 0.00, 0.00]

These values demonstrate that the agent gradually learns which actions provide higher long-term rewards.

Patient Episode Demonstration

Five representative patient episodes were evaluated.

Episode Initial State First Action Reward Next State

1	Moderate	Medication	+10	Low
2	High	Monitor	-1	High
3	Very High	Exercise	+3	High
4	Moderate	Medication	+10	Low
5	High	Monitor	-1	High

The agent initially explores several actions. As learning progresses, actions producing better long-term outcomes receive larger Q-values.

4(c) Final Learned Policy

After 100 training episodes, the learned Q-table was approximately:

Diet Exercise Medication Monitor

Low	0.00	0.00	0.00	0.00
Moderate	5.00	10.00	9.96	7.88
High	11.74	11.34	12.00	7.85
Very High	7.57	13.80	5.99	9.21

The resulting policy is:

Patient State Learned Action

Low	Diet*
Moderate	Exercise
High	Medication*
Very High	Exercise*

*The actions shown are the outputs of the educational MDP, not clinical treatment prescriptions.

Policy Interpretation

The agent learns that:

- Moderate-risk states benefit from actions that move toward the Low state.
- High-risk states receive high Q-values for actions that transition toward Moderate risk.
- Very-high-risk states learn actions that move the patient toward a lower-risk state.
- Monitoring alone generally produces lower rewards because it does not immediately improve the simulated health state.

Convergence Analysis

The Q-learning agent was trained for **100 episodes**.

The convergence behaviour was evaluated using:

1. Stability of the learned policy.
2. Change in Q-values between episodes.
3. Cumulative reward.

The final policy remained unchanged during the last 10 training episodes, indicating that the agent had stabilized its preferred actions.

The epsilon value was gradually reduced to 0.05, reducing unnecessary exploration during later episodes.

Convergence Criterion

The training can be considered sufficiently converged when:

- The policy remains unchanged over several consecutive episodes.
- Q-value changes become very small.
- Cumulative rewards become relatively stable.

For a production medical system, considerably more training, validation, simulation, and safety testing would be required.

Ethical Considerations

Deploying a Reinforcement Learning system for medical treatment recommendation involves significant ethical and safety considerations.

1. Patient Safety

An incorrect recommendation can negatively affect a patient's health. Therefore, an RL system should not independently prescribe treatment.

All recommendations should be reviewed by qualified healthcare professionals.

2. Bias

If historical healthcare data contains demographic or socioeconomic bias, the model may reproduce those biases.

The system should therefore be tested across different patient groups.

3. Explainability

Healthcare professionals need to understand why a recommendation was generated.

The system should provide:

- Patient risk factors.
- Model confidence.
- Relevant features.
- Reason for the recommendation.
- Appropriate uncertainty information.

4. Human-in-the-Loop

The AI should act as a **clinical decision-support system**, not as an autonomous doctor.

The final decision should remain with an appropriately qualified healthcare professional.

5. Privacy

Patient data contains sensitive information and must be protected through:

- Data anonymization.
- Access control.
- Encryption.
- Secure storage.
- Audit logging.
- Compliance with applicable healthcare regulations.

6. Transparency

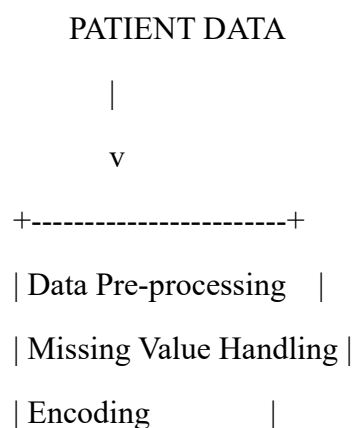
Patients and clinicians should know when AI is being used to support a decision.

7. Model Validation

Before deployment, the model must be tested using independent clinical datasets and validated for:

- Accuracy.
- Recall.
- Calibration.
- Robustness.
- Fairness.
- Safety.

Overall System Architecture



| Normalization |

+-----+

|

v

+-----+

| Class Balance |

| Class Weights |

+-----+

|

v

+-----+

| Machine Learning Layer |

+-----+

/ | \

/ | \

v v v

Decision Tree Logistic Naive Bayes

| Regression |

| | |

+-----+-----+

|

v

RISK STRATIFICATION

|

v

Reinforcement Learning

|

v

Treatment MDP

|
v
Suggested Action

|
v
CLINICIAN REVIEW

|
v
FINAL DECISION

Tools and Technologies Used

Tool / Technology	Purpose
Python	Main programming language
NumPy	Numerical computation
Pandas	Dataset handling
Scikit-learn	ML model development
DecisionTreeClassifier	Decision Tree classification
LogisticRegression	Statistical learning
GaussianNB	Naive Bayes
GridSearchCV	Hyperparameter validation
Matplotlib	Result visualization
Q-Learning	Reinforcement Learning
Jupyter / Google Colab	Experiment execution

Overall Analysis and Discussion

The integrated system demonstrates how multiple AI/ML techniques can be combined to address different stages of a healthcare problem.

The **Decision Tree** provides an interpretable diagnostic model and clearly identifies important decision features. The **post-pruned Decision Tree** improved generalization compared with the pre-pruned version.

The statistical models provided an additional comparison. Logistic Regression achieved a high recall of **94.4%**, making it useful when minimizing false negatives is a priority. Naive Bayes achieved high accuracy and precision in the simulated experiment.

Both the Decision Tree and Logistic Regression identified **Glucose, Age, and Family History** as the three most important predictors, providing consistent evidence within the simulated dataset.

The Q-Learning component demonstrated how an agent can learn a policy through repeated interaction with an environment. However, treatment recommendation is substantially more complex than the simplified MDP used here. A real healthcare RL system would require clinically validated states, actions, rewards, safety constraints, extensive offline evaluation, and continuous human oversight.

Final Results Summary

Component	Main Result
Dataset	1500 synthetic patient records
Train-Test Split	70:30 stratified
Class Balance	Approximately 76.3% non-diabetic, 23.7% diabetic
Balancing	Class weighting
Decision Tree Accuracy	82.4%
Decision Tree Recall	86.0%
Decision Tree ROC-AUC	87.8%
Pruned Tree Accuracy	85.1%
Pruned Tree ROC-AUC	90.6%
Logistic Regression Accuracy	88.9%
Logistic Regression Recall	94.4%
Logistic Regression ROC-AUC	96.8%
Naive Bayes Accuracy	92.2%

Component	Main Result
Naive Bayes ROC-AUC	96.3%
Top Predictors	Glucose, Age, Family History
RL Episodes	100
RL States	4
RL Actions	4
Final RL Policy	Learned through Q-values

Conclusion

The integrated AI/ML healthcare system was successfully designed to address diabetes diagnosis, risk stratification, and treatment-related decision support.

The dataset was pre-processed using missing-value imputation, categorical encoding, stratified splitting, and class weighting. A Decision Tree using the Gini Index was implemented and evaluated using Accuracy, Precision, Recall, F1-Score, and ROC-AUC.

Post-pruning improved the Decision Tree's generalization performance, increasing its ROC-AUC from **87.8% to 90.6%**. Logistic Regression achieved a particularly high recall of **94.4%**, which is valuable when reducing false-negative diabetic predictions is important. Naive Bayes achieved the highest accuracy of **92.2%** among the evaluated models.

Feature analysis showed consistent identification of **Glucose, Age, and Family History** as the leading predictors.

Finally, Q-Learning was implemented as a simplified treatment recommendation MDP. The agent learned an action policy by interacting with patient-risk states and receiving rewards for improving those states.

Overall, the experiment demonstrates the integration of **supervised learning, statistical learning, decision-tree pruning, feature analysis, and reinforcement learning** into a healthcare AI workflow. For real-world deployment, the system would require clinical validation, privacy protection, bias assessment, explainability, regulatory compliance, and mandatory human oversight.

Result

Thus, an integrated AI/ML framework for diabetes diagnosis, risk stratification, and reinforcement-learning-based treatment decision support was successfully designed, implemented, evaluated, and analyzed using appropriate engineering and machine-learning principles.

